

Project: Scalable Web Application with Load Balancing

Services: EC2, ELB, Auto Scaling, RDS, S3, Route 53

Goal: Deploy a web app that scales based on load, with database and static file hosting.

Skills: VPC, security groups, EC2 launch templates, user data scripts.

Definition: This project demonstrates the deployment of a web application that automatically scales based on user traffic, provides a highly available database backend, and hosts static assets (like CSS, JavaScript, and images) on Amazon S3. The solution leverages AWS services such as EC2 for application servers, Auto Scaling Groups to dynamically adjust server capacity, Application Load Balancer (ALB) for distributing traffic across instances, RDS for a managed MySQL database, and S3 for hosting static content.

Scope of the Project:

- Scalable Web App: Auto-scaling EC2 instances behind an Application Load Balancer.
- Database Integration: MySQL database on Amazon RDS with secure connectivity.
- Static File Hosting: CSS, JS, and images hosted on S3.
- Secure Networking: VPC with public/private subnets and proper security groups.
- High Availability & Load Handling: App remains available and responsive under varying load.
- Testing & Monitoring: Verify web app, database, and static content connectivity; test auto-scaling.

Methodology:

1.Manual AWS Console Method (GUI-Based Setup)

- Uses EC2, ALB (ELB), Auto Scaling, RDS, S3, and Route 53.
- Provides hands-on understanding of VPC, security groups, EC2 launch templates, and user data scripts.
- Enables step-by-step configuration of EC2 instances, database setup, static file hosting, and load balancing.
- Allows testing of auto-scaling, database connectivity, and static asset accessibility.
- Ideal for learning, demos, or proof-of-concept deployments with controlled workloads.
- Cost-effective, as resources can be sized small for academic projects.

2.CLI / Script-Based Method (Automated Setup)

- Uses AWS CLI or scripts to create EC2, ALB, Auto Scaling, RDS, S3, and Route 53 resources.
- Ensures consistent and repeatable deployment of infrastructure across multiple environments.
- Reduces manual configuration errors and deployment time.
- Supports automation of web server setup, database connection, and static asset upload using user-data scripts.
- Suitable for professional or multi-environment projects, or where quick redeployment is needed.

AWS Services used in Project:

1.EC2 (Elastic Compute Cloud)

- **Definition:** Virtual servers in the cloud that can run applications or web servers.
- **Purpose:** Host the web application and process client requests.
- **Role in Project:** Runs the web server, serves dynamic content, and connects securely to the RDS database

2.RDS (Relational Database Service)

- **Definition:** Managed relational database service that supports MySQL and other engines.
- **Purpose:** Provide a secure, scalable, and highly available database for the application.
- **Role in Project:** Stores application data; EC2 instances perform read/write operations efficiently.

3. S3 (Simple Storage Service)

- **Definition:** Object storage service for storing and retrieving files in the cloud.
- **Purpose:** Host static content such as CSS, JavaScript, and images.
- **Role in Project:** Stores static assets and serves them to users, reducing the load on EC2 servers.

4.ALB / ELB (Application Load Balancer / Elastic Load Balancer)

- **Definition:** Distributes incoming traffic across multiple EC2 instances to ensure high availability and fault tolerance.
- **Purpose:** Balance user load evenly and prevent any single server from being overloaded.
- **Role in Project:** Routes user requests to EC2 instances efficiently, maintaining consistent performance and uptime.

5. Auto Scaling

- **Definition:** Automatically adjusts the number of EC2 instances based on load and traffic patterns.
- **Purpose:** Maintain application performance while optimizing costs.
- **Role in Project:** Adds or removes EC2 instances as needed, ensuring the application can handle varying traffic.

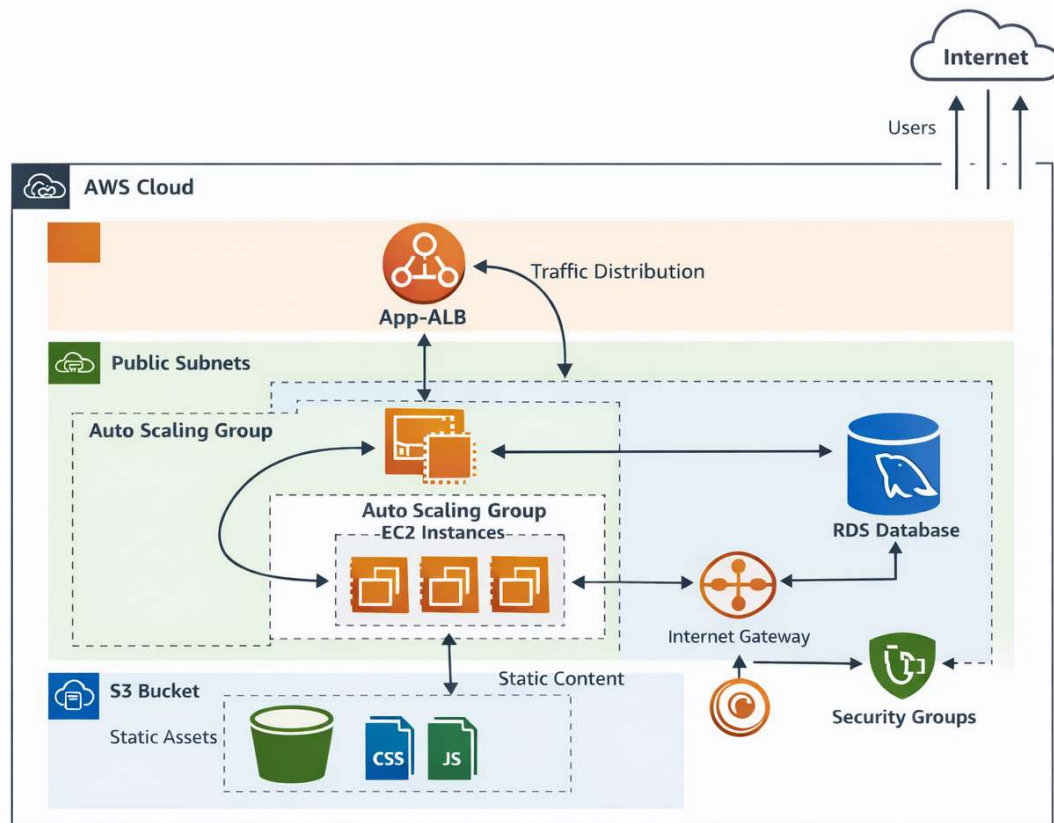
6. VPC (Virtual Private Cloud)

- **Definition:** Isolated virtual network environment in AWS for deploying resources securely.
- **Purpose:** Organize and secure network infrastructure for the application.
- **Role in Project:** Hosts EC2, RDS, and ALB in public and private subnets, controlling access and traffic between components.

7.Security Groups

- **Definition:** Virtual firewalls that control inbound and outbound traffic for AWS resources.
- **Purpose:** Protect EC2, RDS, and other resources by restricting access to only authorized traffic.
- **Role in Project:** Ensures only allowed connections reach the servers and database.

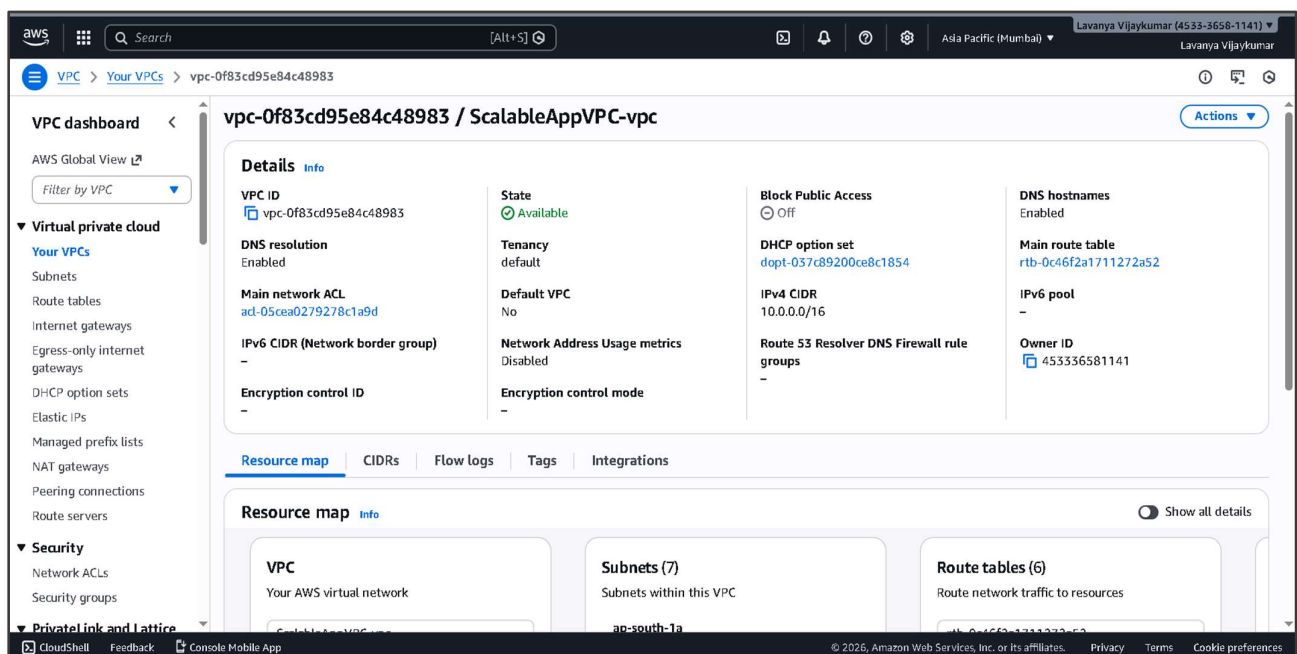
Architecture Diagram:



Project Implementation Steps:

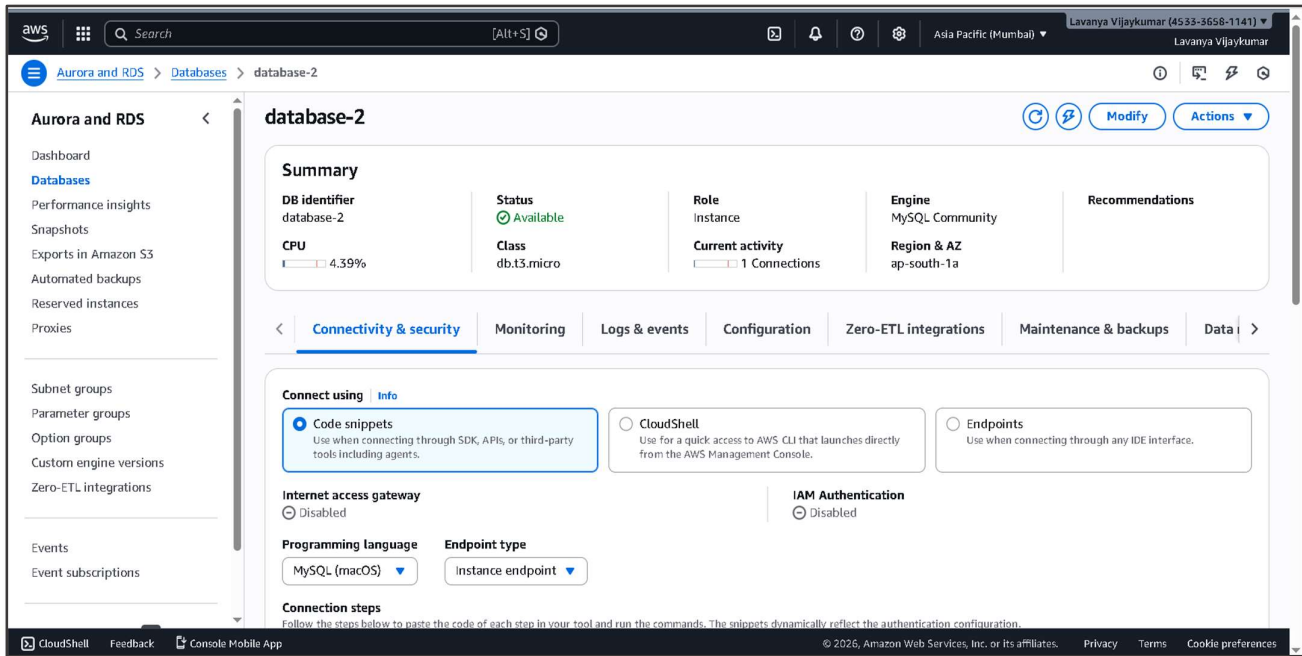
STEP 1: Create VPC and Subnets

- VPC: ScalableAppVPC (CIDR: 10.0.0.0/16)
- Subnets:
 Public subnet: 10.0.1.0/24 → for ALB and public EC2
 Private subnet: 10.0.2.0/24, 10.0.3.0/24 → for EC2 app servers, RDS



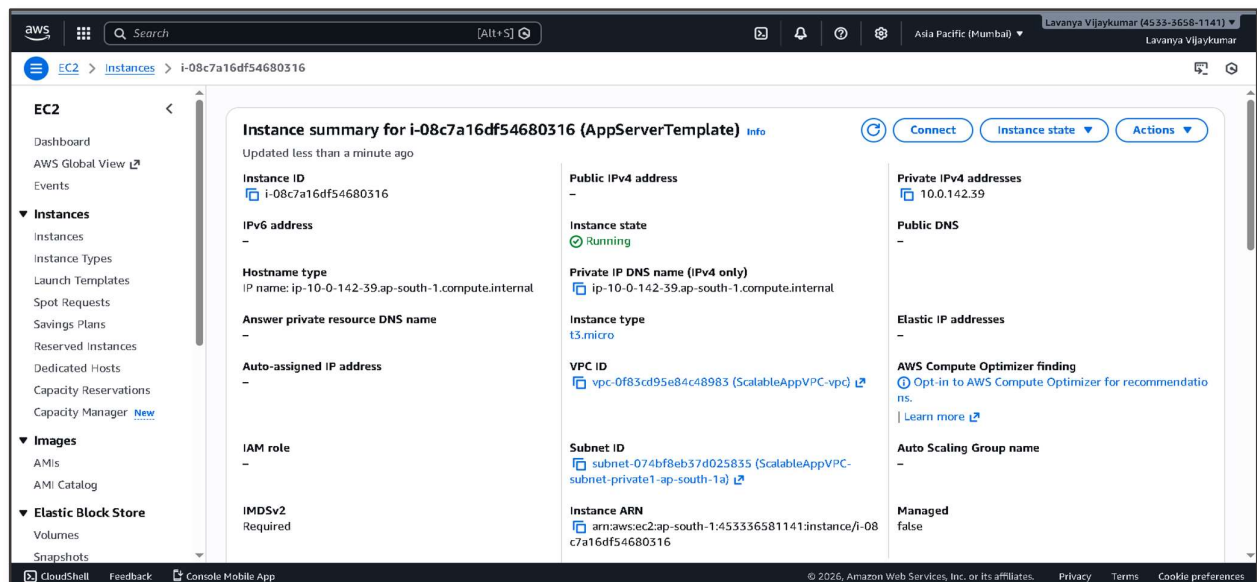
STEP 2: Launch RDS Database

- RDS engine: MySQL 8.0
- DB instance identifier: database-2
- Subnet group: Private subnets
- Security group: DB-SG → allows inbound from EC2 app servers
- Username / Password: admin / Admin_1234567
- Public accessibility: No



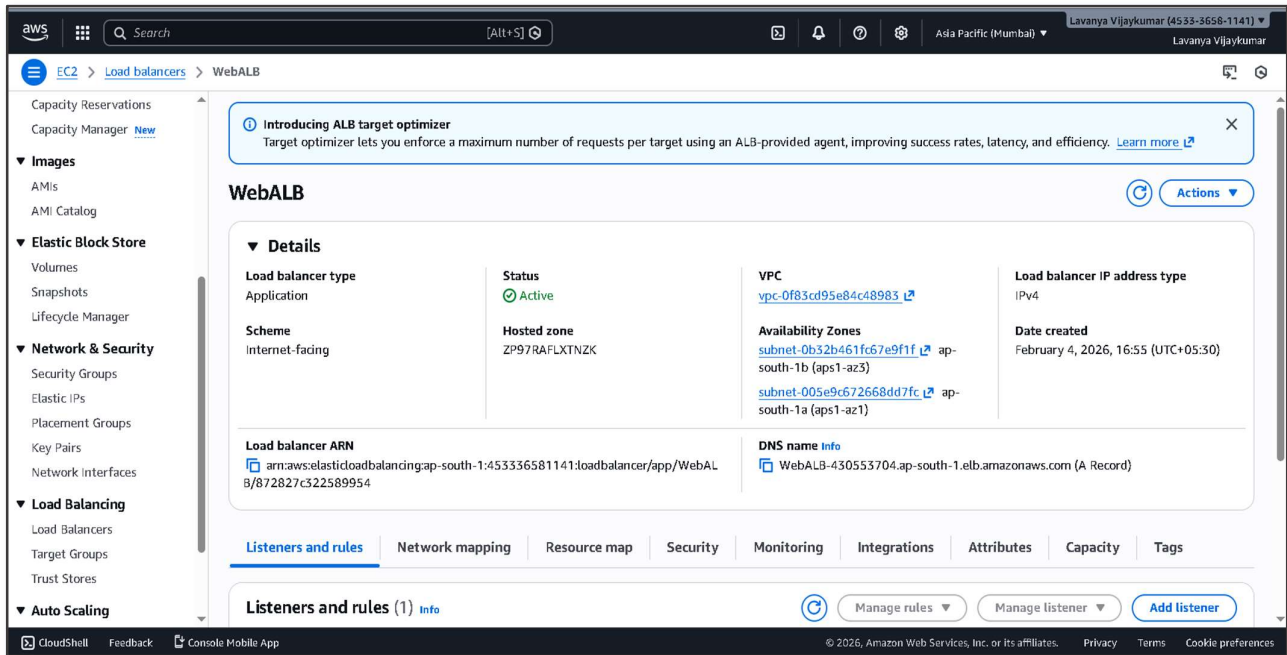
STEP 3: Launch EC2 App Servers

- Launch Template: AppServerTemplate
- AMI: Amazon Linux 2023 / Ubuntu
- Security group: App-SG → allows inbound 80, 22
- Subnet: Private subnets
- User Data: Install web server & connect to RDS



STEP 4: Configure Load Balancer (ALB)

- Create ALB: WebALB
- Subnets: Public subnets
- Security group: Allow HTTP 80, HTTPS 443
- Target group: AppServerTG → register EC2 app servers

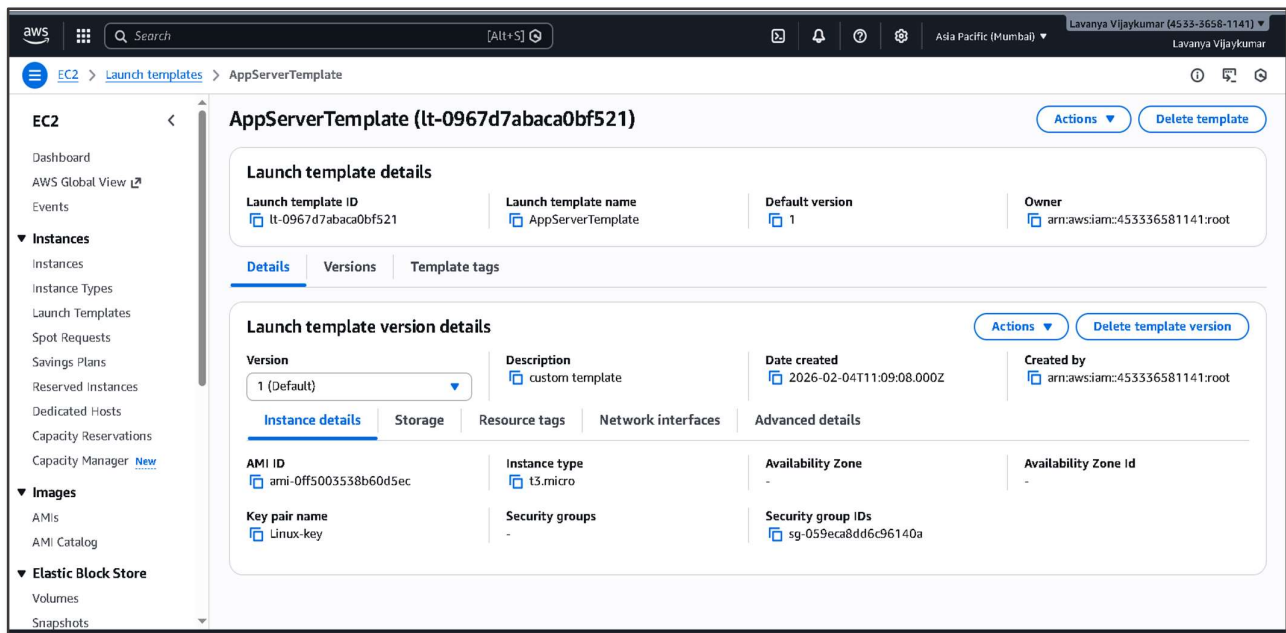


- ALB DNS name visible: WebALB-430553704.ap-south-1.elb.amazonaws.com
- Accessing ALB DNS → HTTP page served by EC2

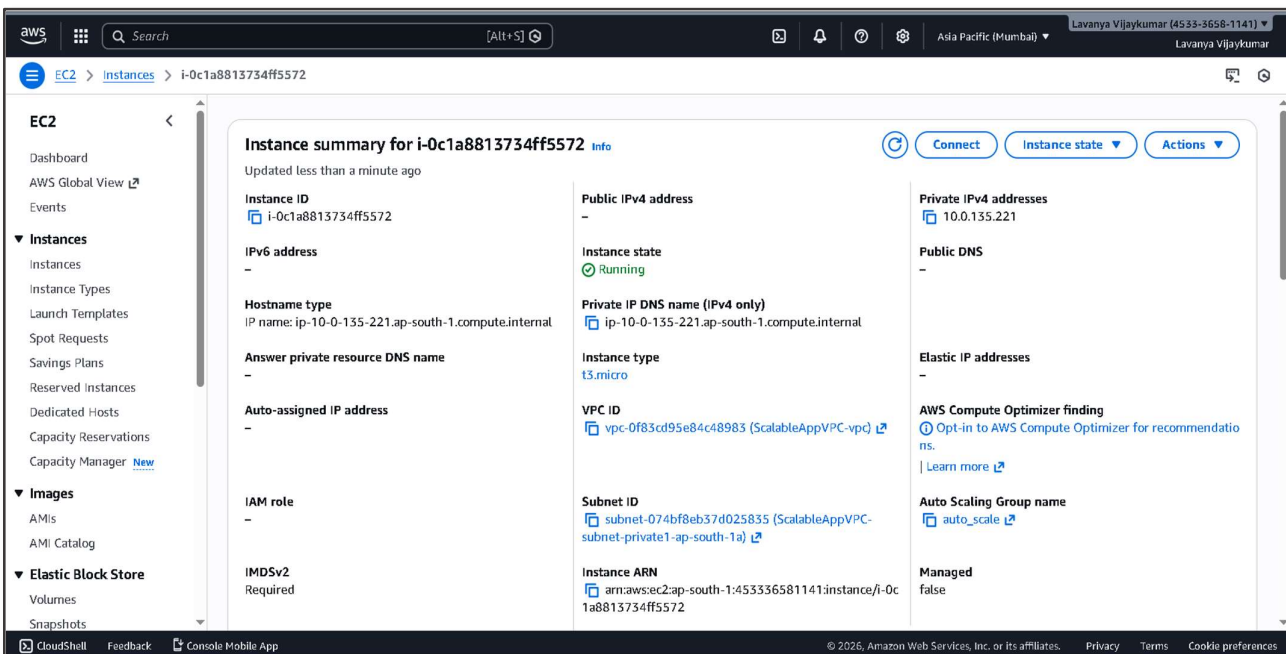


STEP 5: Auto Scaling Group (ASG)

- Launch Template: AppServerTemplate
- Target Group: AppServerTG
- Min / Max instances: 1 / 3
- Scaling Policy: Scale out if CPU > 60%, scale in if CPU < 20%

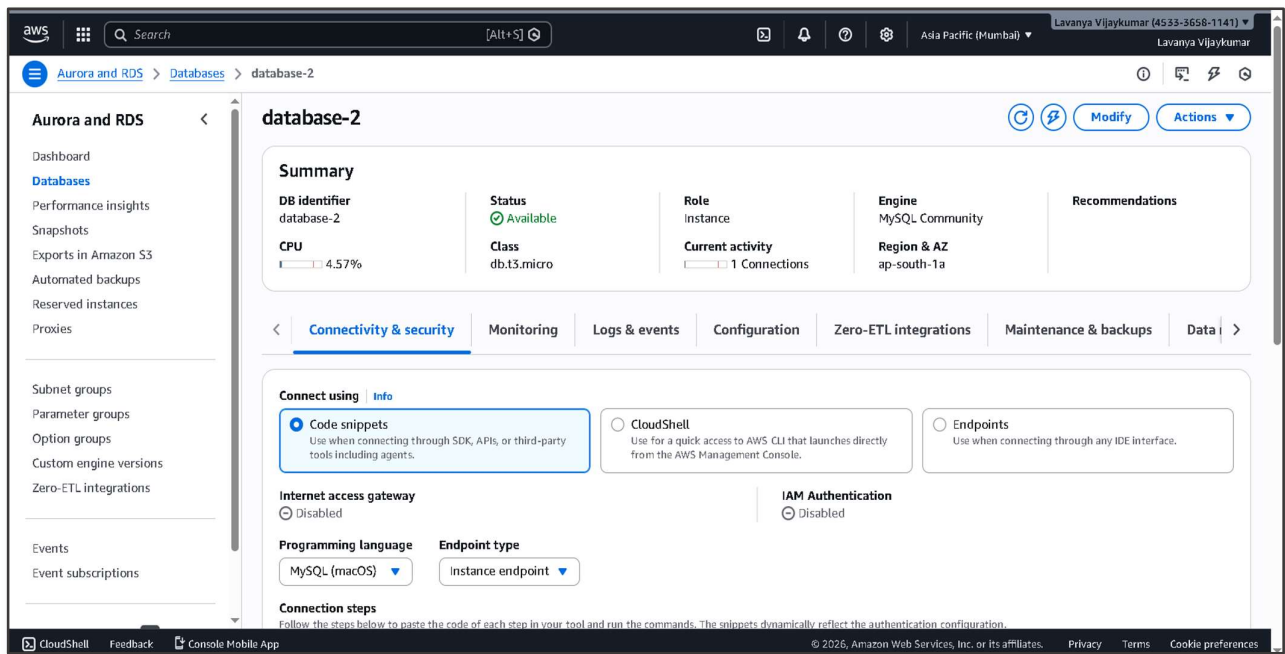


- EC2 instances automatically scale based on load
- ALB distributes traffic to instances



STEP 6: Create RDS Database

- Go to RDS → Create Database → Standard Create
- Engine: MySQL 8.x
- DB Instance Identifier: database-2
- Credentials: Username + Password
- DB Instance Class: db.t2.micro
- Publicly Accessible: No
- VPC: ScalableAppVPC
- Subnet Group: Private1 + Private2
- Security Group: DB-SG



- Connect to RDS from EC2

```
ERROR 2003 (HY000): UNKNOWN MySQL server host: webappdb.ct0qqicmoz93.ap-south-1.rds.amazonaws.com (-2)
[ec2-user@ip-10-0-142-39 ~]$ mysql -h database-2.ct0qqicmoz93.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 63
Server version: 8.4.7 Source distribution

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE webapp;
```

- Create a new database and table

```
mysql> CREATE DATABASE webapp;
ERROR 1007 (HY000): Can't create database 'webapp'; database exists
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| webapp |
+-----+
5 rows in set (0.00 sec)

mysql> USE webapp;
Database changed
mysql> CREATE TABLE users (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   name VARCHAR(50) NOT NULL,
  ->   email VARCHAR(100) NOT NULL UNIQUE,
  ->   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql> INSERT INTO users (name, email) VALUES ('Lavanya V', 'lavu@gmail.com');
Query OK, 1 row affected (0.02 sec)
```

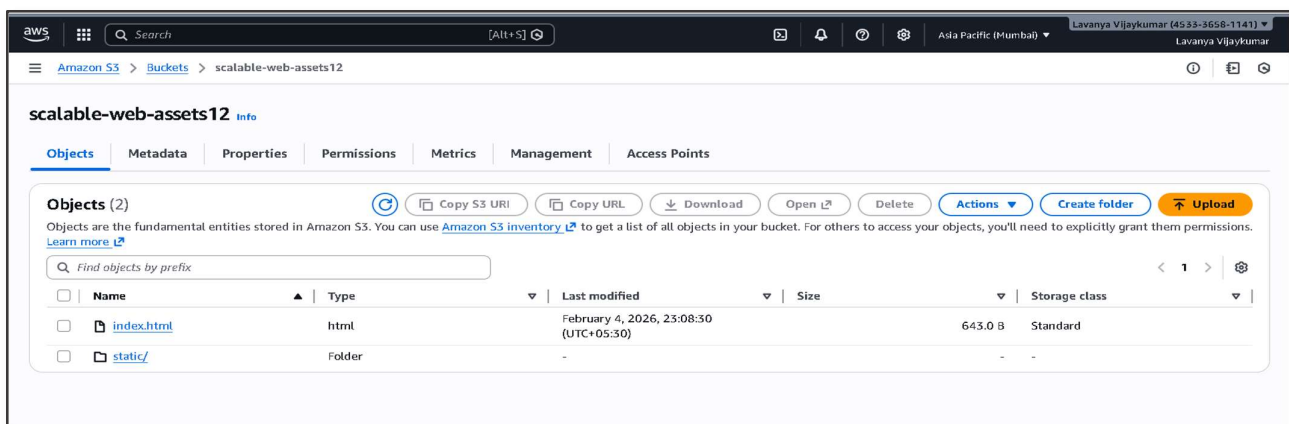

- Verify database and table

```
mysql> SELECT * FROM users;
+-----+-----+-----+-----+
| id | name   | email          | created_at          |
+-----+-----+-----+-----+
| 1  | Lavanya V | lavu@gmail.com | 2026-02-04 17:09:23 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

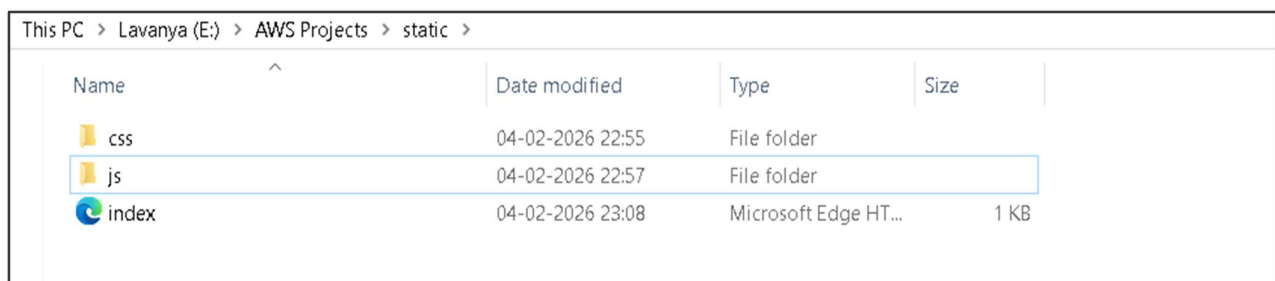
STEP 7: Configure S3 for Static Files

- Create bucket: scalable-web-assets
- Upload CSS, JS.
- Update app code to reference S3 URLs



STEP 8: Connect Web App to RDS and S3

- Create a sample HTML page (index.html) on EC2 or in your project folder



- Static files served from S3



Troubleshooting:**Issue1:** MySQL client not found on EC2

- **Issue:** mysql: command not found
- **Cause:** MySQL client is not installed or missing libraries.
- **Solution:** Install MySQL client depending on OS
sudo yum install -y mysql
sudo dnf install -y mysql

Issue 2: Cannot connect to RDS from EC2

- **Issue:** ERROR 2003 (HY000): Can't connect to MySQL server
- **Cause:** EC2 and RDS are in different subnets or VPCs.
- **Solution:** Ensure EC2 and RDS are in the same VPC.

Conclusion:

This project demonstrates the deployment of a highly scalable and resilient web application using AWS cloud services. By integrating EC2, RDS, S3, ALB, and Auto Scaling, the application can dynamically handle varying traffic loads while maintaining performance and availability. The RDS database ensures reliable storage of application data, and S3 efficiently serves static files, reducing load on the web servers. Auto Scaling and Load Balancing guarantee that the system can automatically scale out during peak traffic and scale in when demand decreases, optimizing cost and resource utilization.

Through this project, practical skills in VPC setup, security groups, web server configuration, database management, and cloud resource orchestration were gained. The solution provides a blueprint for building cloud-native, scalable web applications with high availability, fault tolerance, and efficient resource management.